



# **CS2009**

# **DESIGN AND ANALYSIS OF ALGORITHMS**

**Dr. Hashim Yasin**

**National University of Computer  
and Emerging Sciences,  
Faisalabad, Pakistan.**

SORTING



# Algorithm

3

- An algorithm is a well-defined computational procedure that takes some value, or set of values, as **input** and produces some value, or set of values, as **output**.

***For Example:***

**Input:** A sequence of  $n$  numbers  $\langle a_1, a_2, \dots, a_n \rangle$ .

**Output:** A permutation (reordering)  $\langle a'_1, a'_2, \dots, a'_n \rangle$  of the input sequence such that  $a'_1 \leq a'_2 \leq \dots \leq a'_n$ .

# Sorting

4

**Input:** A sequence of  $n$  numbers  $\langle a_1, a_2, \dots, a_n \rangle$ .

**Output:** A permutation (reordering)  $\langle a'_1, a'_2, \dots, a'_n \rangle$  of the input sequence such that  $a'_1 \leq a'_2 \leq \dots \leq a'_n$ .

**For example,**

- ▣ given the input sequence (31; 41; 59; 26; 41; 58),
- ▣ a sorting algorithm
- ▣ returns as output sequence (26; 31; 41; 41; 58; 59).
- Such an input sequence is called an ***instance*** of the sorting problem

# INSERTION SORT



# Insertion Sort

6

| INSERTION-SORT( <i>A</i> )                                                | <i>cost</i> | <i>times</i>             |
|---------------------------------------------------------------------------|-------------|--------------------------|
| 1 <b>for</b> <i>j</i> = 2 <b>to</b> <i>A.length</i>                       | $c_1$       | $n$                      |
| 2 $key = A[j]$                                                            | $c_2$       | $n - 1$                  |
| 3       // Insert $A[j]$ into the sorted<br>sequence $A[1 \dots j - 1]$ . | 0           | $n - 1$                  |
| 4 $i = j - 1$                                                             | $c_4$       | $n - 1$                  |
| 5 <b>while</b> $i > 0$ and $A[i] > key$                                   | $c_5$       | $\sum_{j=2}^n t_j$       |
| 6 $A[i + 1] = A[i]$                                                       | $c_6$       | $\sum_{j=2}^n (t_j - 1)$ |
| 7 $i = i - 1$                                                             | $c_7$       | $\sum_{j=2}^n (t_j - 1)$ |
| 8 $A[i + 1] = key$                                                        | $c_8$       | $n - 1$                  |

---

$$\begin{aligned} T(n) = & c_1 n + c_2(n - 1) + c_4(n - 1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) \\ & + c_7 \sum_{j=2}^n (t_j - 1) + c_8(n - 1) . \end{aligned}$$

# Insertion Sort (Best Case)

7

$$T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) + c_7 \sum_{j=2}^n (t_j - 1) + c_8(n-1).$$

**Best Case:**

|   |   |   |   |   |
|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|

$$\begin{aligned} T(n) &= c_1n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1) \\ &= (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8). \end{aligned}$$

It is thus a linear function of  $n$

$$T(n) = O(n)$$

# Insertion Sort (Worst Case)

8

$$T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) \\ + c_7 \sum_{j=2}^n (t_j - 1) + c_8(n-1).$$

**Worst Case:**

|   |   |   |   |   |
|---|---|---|---|---|
| 5 | 4 | 3 | 2 | 1 |
|---|---|---|---|---|

$$T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5 \left( \frac{n(n+1)}{2} - 1 \right) \\ + c_6 \left( \frac{n(n-1)}{2} \right) + c_7 \left( \frac{n(n-1)}{2} \right) + c_8(n-1) \\ = \left( \frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2} \right) n^2 + \left( c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8 \right) n \\ - (c_2 + c_4 + c_5 + c_8).$$

$T(n) = O(n^2)$

# Insertion Sort

9

## □ Best case:

elements already sorted,  $t_i = 1$ ,  
running time =  $f(n)$ , i.e., *linear time*.

## □ Worst case:

elements are sorted in inverse order,  
 $t_i = i$ , running time =  $f(n^2)$ , i.e., *quadratic time*

## □ Average case:

$t_i = i/2$ , running time =  $f(n^2)$ ,  
i.e., *quadratic time*

# Review on Summations

10

```
MaxSubvector(A, n)
    maxsum ← 0;
    for i ← 1 to n
        for j = i to n
            sum ← 0
            for k ← i to j
                sum += A[k]
            maxsum ← max(sum, maxsum)
    return maxsum
```

$$T(n) = \sum_{i=1}^n \sum_{j=i}^n \sum_{k=i}^j 1$$

**NOTE:** This is not a simplified solution. What is the final answer?

# Review on Summations

- **Constant Series:** For integers  $a$  and  $b$ ,  $a \leq b$ ,

$$\sum_{i=a}^b 1 = b - a + 1$$

- **Linear Series (Arithmetic Series):** For  $n \geq 0$ ,

$$\sum_{i=1}^n i = 1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

- **Quadratic Series:** For  $n \geq 0$ ,

$$\sum_{i=1}^n i^2 = 1^2 + 2^2 + \cdots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

# Review on Summations

12

- **Cubic Series:** For  $n \geq 0$ ,

$$\sum_{i=1}^n i^3 = 1^3 + 2^3 + \cdots + n^3 = \frac{n^2(n+1)^2}{4}$$

- **Geometric Series:** For real  $x \neq 1$ ,

$$\sum_{k=0}^n x^k = 1 + x + x^2 + \cdots + x^n = \frac{x^{n+1} - 1}{x - 1}$$

# Review on Summations

13

□ **Linear-Geometric Series:** For  $n \geq 0$ , real  $c \neq 1$ ,

$$\sum_{i=1}^n i c^i = c + 2c^2 + \cdots + n c^n = \frac{-(n+1)c^{n+1} + n c^{n+2} + c}{(c-1)^2}$$

# BUBBLE SORT



# Standard (Unoptimized) Bubble Sort

```
int n = N; // N is the size of the array;
for (int i = 0; i < N; i++){
    for (int j = 1; j < n; j++) {
        if (A[j] < A[j-1]) {
            swap(j-1, j);
        } //end if
    } //end inner for
    n = n-1;
} //end outer for
```

```
// Swap function assumes that
// A[n] is a globally declared array
swap ( x , y ) {
    int temp = A[x];
    A[x] = A[y];
    A[y] = temp;
}
```



Algorithm does not exit  
until all the data is checked

Complexity ?       $O(N^2)$

# Optimized Bubble Sort

```
int n = N; // N is the size of the array;
for (int i = 0; i < N; i++){
    int swapped = 0;
    for (int j = 1; j < n; j++) {
        if (A[j] < A[j-1]) {
            swap(j-1, j);
            swapped = 1;
        } //end if
    } //end inner for
    n = n-1;
    if (swapped == 0)
        break;
} //end outer for
```

```
// Swap function assumes that
// A[n] is a globally declared array
swap ( x , y) {
    int temp = A[x];
    A[x] = A[y];
    A[y] = temp;
}
```



Algorithm exits if no swap done  
in previous (outer loop) step

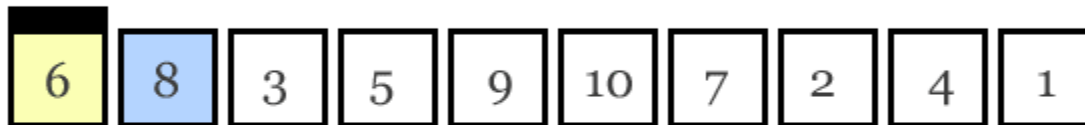
Complexity ?  $O(N^2)$

# SELECTION SORT



# Selection Sort

18



Yellow is smallest number found

Blue is current item

Green is sorted list

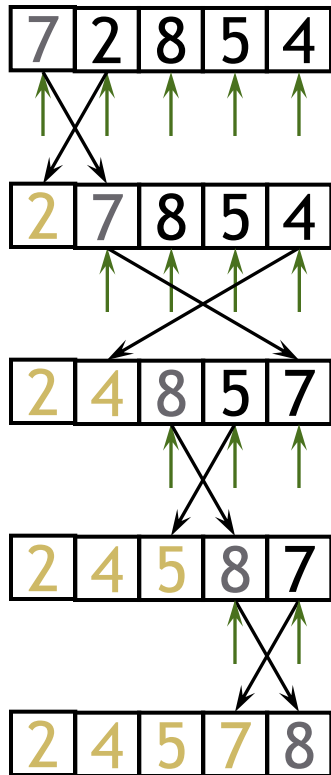
# Selection Sort

```
for (int i = 0, i < n, i++) {  
    min = i;  
    for (int j = i + 1, j < n, j++) {  
        if (A[j] < A[min]) {  
            min = j  
        } // end if  
    } // end inner for  
    swap(i, min)  
} // end outer for
```

Complexity ?  
 $O(N^2)$

```
// Swap function assumes that  
// A[n] is a globally declared array  
swap(i, min) {  
    int temp = A[i];  
    A[i] = A[min];  
    A[min] = temp;  
}
```

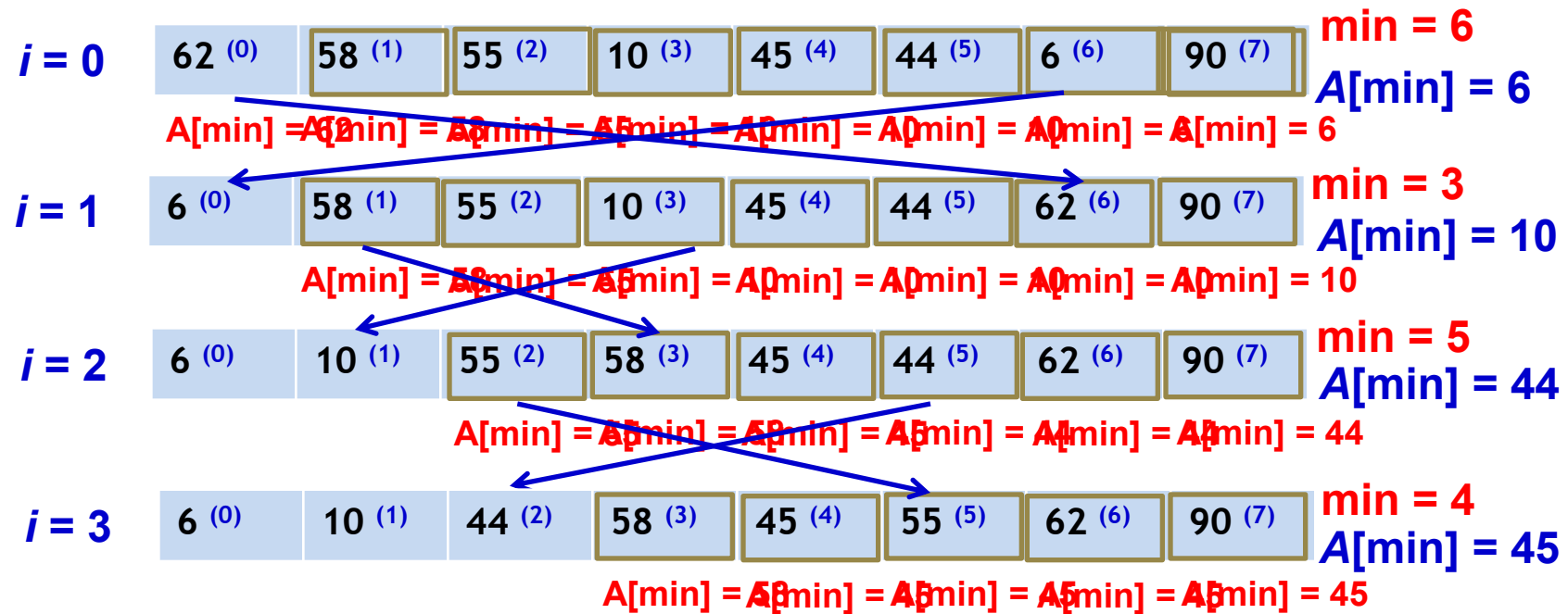
# Selection Sort - Example



- The Selection Sort might swap an array element with itself--this is harmless.

```
for (int i=0, i < n, i++){  
    min = i;  
    for (int j = i+1, j < n, j++){  
        if ( A[j] < A[min]){  
            min = j  
        } // end if  
    } // end inner for  
    swap(i, min)  
} // end outer for
```

# Selection Sort Example



```

for (int i=0, i < n, i++){
    min = i;
    for (int j = i+1, j < n, j++){
        if ( A[j] < A[min]){
            min = j
        } // end if
    } // end inner for
    swap(i, min)
} // end outer for
    
```

# Selection Sort Example - continued

|         |                   |                   |                   |                   |                   |                   |                   |                   |                                          |
|---------|-------------------|-------------------|-------------------|-------------------|-------------------|-------------------|-------------------|-------------------|------------------------------------------|
| $i = 0$ | 62 <sup>(0)</sup> | 58 <sup>(1)</sup> | 55 <sup>(2)</sup> | 10 <sup>(3)</sup> | 45 <sup>(4)</sup> | 44 <sup>(5)</sup> | 6 <sup>(6)</sup>  | 90 <sup>(7)</sup> | $\text{min} = 6$<br>$A[\text{min}] = 6$  |
| $i = 1$ | 6 <sup>(0)</sup>  | 58 <sup>(1)</sup> | 55 <sup>(2)</sup> | 10 <sup>(3)</sup> | 45 <sup>(4)</sup> | 44 <sup>(5)</sup> | 62 <sup>(6)</sup> | 90 <sup>(7)</sup> | $\text{min} = 3$<br>$A[\text{min}] = 10$ |
| $i = 2$ | 6 <sup>(0)</sup>  | 10 <sup>(1)</sup> | 55 <sup>(2)</sup> | 58 <sup>(3)</sup> | 45 <sup>(4)</sup> | 44 <sup>(5)</sup> | 62 <sup>(6)</sup> | 90 <sup>(7)</sup> | $\text{min} = 5$<br>$A[\text{min}] = 44$ |
| $i = 3$ | 6 <sup>(0)</sup>  | 10 <sup>(1)</sup> | 44 <sup>(2)</sup> | 58 <sup>(3)</sup> | 45 <sup>(4)</sup> | 55 <sup>(5)</sup> | 62 <sup>(6)</sup> | 90 <sup>(7)</sup> | $\text{min} = 4$<br>$A[\text{min}] = 45$ |
| $i = 4$ | 6 <sup>(0)</sup>  | 10 <sup>(1)</sup> | 44 <sup>(2)</sup> | 45 <sup>(3)</sup> | 58 <sup>(4)</sup> | 55 <sup>(5)</sup> | 62 <sup>(6)</sup> | 90 <sup>(7)</sup> | $\text{min} = 5$<br>$A[\text{min}] = 55$ |
| $i = 5$ | 6 <sup>(0)</sup>  | 10 <sup>(1)</sup> | 44 <sup>(2)</sup> | 45 <sup>(3)</sup> | 55 <sup>(4)</sup> | 58 <sup>(5)</sup> | 62 <sup>(6)</sup> | 90 <sup>(7)</sup> | $\text{min} = 5$<br>$A[\text{min}] = 58$ |
| $i = 6$ | 6 <sup>(0)</sup>  | 10 <sup>(1)</sup> | 44 <sup>(2)</sup> | 45 <sup>(3)</sup> | 55 <sup>(4)</sup> | 58 <sup>(5)</sup> | 62 <sup>(6)</sup> | 90 <sup>(7)</sup> | $\text{min} = 6$<br>$A[\text{min}] = 62$ |
| $i = 7$ | 6 <sup>(0)</sup>  | 10 <sup>(1)</sup> | 44 <sup>(2)</sup> | 45 <sup>(3)</sup> | 55 <sup>(4)</sup> | 58 <sup>(5)</sup> | 62 <sup>(6)</sup> | 90 <sup>(7)</sup> | $\text{min} = 6$<br>$A[\text{min}] = 62$ |

```

for (int i=0, i < n, i++){
    min = i;
    for (int j = i+1, j < n, j++){
        if (A[j] < A[min]){
            min = j;
        } // end if
    } // end inner for
    swap(i, min);
} // end outer for

```

# Selection Sort

23

- *Selection Sort has a time complexity of  $O(n^2)$ .*

| Best case | Average case | Worst case |
|-----------|--------------|------------|
| $O(n^2)$  | $O(n^2)$     | $O(n^2)$   |

## Selection Sort vs Bubble Sort

- *Selection Sort selects the minimum element and places it in the correct position with fewer swaps, while*
- *Bubble Sort repeatedly swaps adjacent elements to sort the array.*

# Acknowledgement

24

## **Introduction to Algorithms**

Thomas H. Cormen, Charles E. Leiserson,  
Ronald L. Rivest, and Clifford Stein